INFORME DE VULNERABILIDADES Y CORRECCIONES

Aplicación pnkSecurity (PHP + MySQL) — Desarrollo Seguro

Ítem	Detalle
Fecha	17/10/2026
Integrantes	Matías Zepeda Arrieta · Joaquín Andrés Zambra Zúñiga
Docente	Jorge Cortés
Repositorio	https://github.com/matias725/desarrolo-seguro-
Código vulnerable	rama main (commit 8d54176)
Código corregido	rama mejoras (un commit por grupo de vulnerabilidades)
Versión en AWS	rama mejoras — versión sin vulnerabilidades (sección 5)
Hallazgos	34 (9 Críticos · 5 Altos · 1 Media-Alta · 9 Medios · 10 Bajos) — todos corregidos y verificados en AWS

1. Resumen ejecutivo

Se auditó la aplicación pnkSecurity (menú digital de restaurantes) mediante revisión manual de código (caja blanca) y pruebas dinámicas que repiten cada ataque contra la versión original y la corregida. La primera auditoría identificó 22 hallazgos (informe_pentesting_inicial.docx); una segunda revisión sobre la versión corregida y su despliegue agregó 5 más (VUL-023 a VUL-027) y una tercera revisión completa (código, librerías publicadas y aprovisionamiento de AWS) agregó 6 más (VUL-028 a VUL-033). El testing en AWS del 17/10/2026 detectó una más (VUL-034).

Las 34 vulnerabilidades quedaron corregidas en la rama mejoras y la versión publicada en AWS pasa las 9 pruebas automáticas (17/10/2026). VUL-001 a VUL-026 tienen evidencia de prueba (el ataque falla en la versión corregida). VUL-027 a VUL-033 se verifican con tests/test_seguridad.py (9 pruebas automáticas) y con la configuración versionada del despliegue, y se re-evidencian en cada despliegue con deploy/desplegar_y_evidenciar.ps1.

Causa raíz: ausencia sistemática de consultas preparadas, de validación de tipos y de codificación de salida. La corrección se hizo de forma transversal: setup.php concentra una única función de acceso a datos con consultas preparadas, la validación de enteros, la codificación HTML, los tokens CSRF, la sesión segura y las cabeceras HTTP.

Estructura de cada hallazgo

Cada vulnerabilidad de la sección 3 se documenta con:

- Vulnerabilidad — nombre y severidad
- OWASP — categoría Top 10:2021 y capítulo ASVS v5.0.0
- Descripción
- URL — endpoint o recurso afectado
- Evidencia — ataque y resultado sobre la versión original
- Corrección — commit Git, código original y código nuevo
- Evidencia sin vulnerabilidad — mismo ataque sobre la versión corregida

2. Resumen de hallazgos

ID	Vulnerabilidad	OWASP	Severidad	Commit	Estado
VUL-001	SQL Injection en login (bypass de autenticación)	A03:2021	**Crítica**	f6070c8, 94ef3a9	Corregido – verificado
VUL-002	SQL Injection en index.php – datos del restaurante	A03:2021	**Crítica**	294250b	Corregido – verificado
VUL-003	SQL Injection en index.php – ítems destacados	A03:2021	**Crítica**	294250b	Corregido – verificado
VUL-004	SQL Injection en index.php – listado de cartas y categorías	A03:2021	**Crítica**	294250b	Corregido – verificado
VUL-005	SQL Injection en index.php – comentarios	A03:2021	**Crítica**	294250b, dd0c4f8	Corregido – verificado
VUL-006	SQL Injection en mostrar_carrito.php	A03:2021	**Crítica**	294250b	Corregido – verificado
VUL-007	SQL Injection en carrito.php (AJAX)	A03:2021	**Crítica**	31c4312	Corregido – verificado
VUL-008	SQL Injection en grcomentarios.php (INSERT)	A03:2021	**Crítica**	e964933	Corregido – verificado
VUL-009	Contraseñas almacenadas en texto plano	A02:2021	**Crítica**	f6070c8	Corregido – verificado
VUL-010	Cross-Site Scripting (XSS) almacenado en comentarios	A03:2021	**Alta**	294250b, 94ef3a9	Corregido – verificado
VUL-011	Ausencia de token CSRF en el login	A01:2021	**Alta**	f6070c8, 294250b, 94ef3a9	Corregido – verificado
VUL-012	Ausencia de token CSRF en comentarios	A01:2021	**Alta**	e964933,	Corregido – verificado

				294250b	
VUL-013	Ausencia de token CSRF en el carrito (AJAX)	A01:2021	Media	31c4312	Corregido – verificado
VUL-014	Autorización solo en el cliente (comentarios sin sesión)	A01:2021	Alta	e964933	Corregido – verificado
VUL-015	IDOR en el parámetro id (restaurante)	A01:2021	Alta	294250b	Corregido – verificado
VUL-016	Lógica de negocio: agregar ítems ocultos/eliminados al carrito	A04:2021	Media	31c4312	Corregido – verificado
VUL-017	Fijación de sesión (session fixation)	A07:2021	Media	f6070c8, 94ef3a9	Corregido – verificado
VUL-018	Sin límite de intentos de login (fuerza bruta)	A07:2021	Media	f6070c8, d57a953	Corregido – verificado
VUL-019	Credenciales de BD embebidas en el código (root sin clave)	A05:2021	Media	94ef3a9, 3b4bca9	Corregido – verificado
VUL-020	Validación de entradas insuficiente (causa raíz transversal)	A03:2021	Media	94ef3a9, 294250b, 31c4312	Corregido – verificado
VUL-021	Errores técnicos expuestos en pantalla	A05:2021	Baja	94ef3a9	Corregido – verificado
VUL-022	Faltan cabeceras de seguridad y atributos de cookie	A05:2021	Baja	94ef3a9	Corregido – verificado
VUL-023	Credenciales de prueba conocidas en producción y secretos en log	A07:2021	Media-Alta	534932c	Corregido – verificado en AWS (17/10/2026)
VUL-024	Bloqueo de cuentas ajenas (DoS) en el límite de intentos	A07:2021	Media	d57a953	Corregido – verificado
VUL-025	Componentes JavaScript con vulnerabilidades conocidas	A06:2021	Baja	273af11	Corregido – verificado
VUL-026	Comentarios sin límite (spam / inundación)	A04:2021	Baja	dd0c4f8	Corregido – verificado
VUL-027	Versión vulnerable expuesta indefinidamente en :8080	A05:2021	Baja	534932c, c617a06	Corregido – verificado en AWS (17/10/2026)
VUL-028	Detección de HTTPS confiando en X-Forwarded-Proto del cliente	A05:2021	Baja	e534271	Corregido – verificar con test_seguridad.py
VUL-029	CSP con 'unsafe-inline' y fuentes externas	A05:2021	Baja	e534271	Corregido – verificado en AWS (17/10/2026)
VUL-030	Librerías obsoletas y archivos sin uso publicados en el webroot	A06:2021	Media	e71f454	Corregido – verificado en AWS (17/10/2026)
VUL-031	Fuerza bruta distribuida contra una cuenta y retención indefinida de IPs	A07:2021	Baja	e3b4e54	Corregido – verificado por revisión de código
VUL-032	Registro insuficiente de eventos de seguridad	A09:2021	Baja	e534271, e3b4e54	Corregido – verificado en AWS (17/10/2026)
VUL-033	Aplicación publicada por HTTP plano si falla el certificado TLS	A02:2021	Media	c617a06	Corregido – verificado en AWS (17/10/2026)
VUL-034	Cabeceras de seguridad duplicadas (Apache + PHP)	A05:2021	Baja	62c29e9	Corregido – verificado en AWS (17/10/2026)

Comparar el código completo: https://github.com/matias725/desarrolo-seguro-/compare/main...mejoras

3. Detalle de vulnerabilidades

URL base — versión original (rama main): http://174.129.56.84:8080 · versión corregida (rama mejoras): https://174-129-56-84.sslip.io. En el laboratorio local: original http://127.0.0.1:8080 y corregida http://127.0.0.1:8000 (tests/lab_local.sh).

VUL-001 → SQL Injection en login (bypass de autenticación)

Vulnerabilidad	SQL Injection en login (bypass de autenticación)
Severidad	**Crítica**
OWASP Top 10	A03:2021 – Injection
OWASP ASVS v5.0.0	V1 Encoding and Sanitization (prevención de inyección); V6 Authentication
Descripción	procesalogin.php concatena frmusuario y frmpassword dentro de la sentencia SQL. Con el payload ' OR '1'='1' -- - la condición WHERE siempre es verdadera y el sistema abre sesión sin conocer ninguna credencial; también permite extraer datos con UNION o técnicas booleanas/tiempo.
URL	POST /setup/procesalogin.php (campo frmusuario)
Archivo(s)	src/pnkSecurity/setup/procesalogin.php
ISO/IEC 27001:2022	A.8.28, A.8.29, A.8.5
ISO/IEC 27034 (ASC)	ASC-07, ASC-10, ASC-14
Estado	Corregido – verificado

Evidencia (versión original — vulnerable)

```
Payload en frmusuario:  ' OR '1'='1' -- -     frmpassword: x
Resultado en la versión original (main): SESIÓN ABIERTA sin credenciales
(la barra superior muestra "Bienvenido :Administrador").
```

Corrección

Commit Git: f6070c8 · 94ef3a9 (rama mejoras)

Código original

```
$sql="select * from usuarios where email='".$_POST['frmusuario']."'
      and password='".$_POST['frmpassword']."'";
$result=mysqli_query(conectar(),$sql);
$cont_login=mysqli_num_rows($result);
$datos=mysqli_fetch_array($result);
if($cont_login!=0) { session_start(); $_SESSION['nombre']=$datos['nombre']; }
```

Código nuevo

```
$usuario = consulta_fila(
    "SELECT Id, nombre, password, estado FROM usuarios WHERE email = ? LIMIT 1",
    's', $email
);
// Se verifica siempre un hash para no revelar qué correos existen.
$hash   = $usuario['password'] ?? '$2y$12$FpiqU2XFu81IFmQVzX5h4O...';
$valido = password_verify($password, $hash) && $usuario !== null
          && $usuario['estado'] === '1';
```

Evidencia sin vulnerabilidad (versión corregida)

```
Mismo payload contra la versión corregida (mejoras): sin sesión (bloqueado).
  [PASA] VUL-1  Login SQLi -> sin sesión (bloqueado)
```

VUL-002 → SQL Injection en index.php – datos del restaurante

Vulnerabilidad	SQL Injection en index.php – datos del restaurante
Severidad	**Crítica**
OWASP Top 10	A03:2021 – Injection
OWASP ASVS v5.0.0	V1 Encoding and Sanitization; V2 Validation
Descripción	$_GET['id'] se asigna a $key sin validación y se concatena en la consulta que carga el restaurante. Cualquier visitante puede alterar la sentencia desde la URL.
URL	GET /index.php?id=0 UNION SELECT 1-- -
Archivo(s)	src/pnkSecurity/index.php
ISO/IEC 27001:2022	A.8.28, A.8.29
ISO/IEC 27034 (ASC)	ASC-10, ASC-14
Estado	Corregido – verificado

Evidencia (versión original — vulnerable)

```
original  index.php?id=0 UNION SELECT 1-- -  -> HTTP 200 (inyección aceptada)
```

Corrección

Commit Git: 294250b (rama mejoras)

Código original

```
$key=$_GET['id'];
$_SESSION['id']=$_GET['id'];
$sql_restorant="SELECT ... FROM restautantes INNER JOIN direcciones ...
  WHERE restautantes.id = ".$key." AND restautantes.eliminado IS NULL";
$result_restorant=mysqli_query(conectar(),$sql_restorant);
```

Código nuevo

```
$key = entero_positivo($_GET['id'] ?? null);   // solo ^[0-9]{1,9}$ y > 0
if ($key === false) {
    responder_error(404, 'Restaurante no encontrado.');
}
$datos_restorant = consulta_fila(
    "SELECT ... FROM restautantes INNER JOIN direcciones ...
     WHERE restautantes.id = ? AND restautantes.eliminado IS NULL",
    'i', $key
);
```

Evidencia sin vulnerabilidad (versión corregida)

```
corregida index.php?id=0 UNION SELECT 1-- -  -> HTTP 404 (rechazado)
  [PASA] VUL-2  SQLi en id -> HTTP 404 (rechazado)
```

VUL-003 → SQL Injection en index.php – ítems destacados

Vulnerabilidad	SQL Injection en index.php – ítems destacados
Severidad	**Crítica**
OWASP Top 10	A03:2021 – Injection
OWASP ASVS v5.0.0	V1 Encoding and Sanitization
Descripción	La consulta de ítems destacados reutiliza $key (derivada de $_GET['id']) concatenada en el WHERE. Es un segundo punto de inyección independiente dentro del mismo archivo.
URL	GET /index.php?id=<payload>
Archivo(s)	src/pnkSecurity/index.php
ISO/IEC 27001:2022	A.8.28
ISO/IEC 27034 (ASC)	ASC-10, ASC-14
Estado	Corregido – verificado

Evidencia (versión original — vulnerable)

```
Código vulnerable (revisión de código, informe inicial):
... AND cartas.restautantes_id = ".$key." AND cartas.eliminada IS NULL ...
Mismo vector que VUL-002: HTTP 200 con payload UNION en la versión original.
```

Corrección

Commit Git: 294250b (rama mejoras)

Código original

```
$sql="SELECT items.id, ... FROM categorias INNER JOIN items ... INNER JOIN cartas ...
  WHERE items.destacado = 1 AND items.eliminado IS NULL AND items.visible=1
  AND cartas.restautantes_id = ".$key." AND cartas.eliminada IS NULL ...";
$result=mysqli_query(conectar(),$sql);
```

Código nuevo

```
// VUL003: consulta preparada.
$destacados_lista = consulta_todas(
    "SELECT items.id, items.tiempo, ... FROM categorias INNER JOIN items ...
     WHERE items.destacado = 1 AND items.eliminado IS NULL AND items.visible = 1
       AND cartas.restautantes_id = ? AND cartas.eliminada IS NULL ...",
    'i', $key
);
```

Evidencia sin vulnerabilidad (versión corregida)

```
El id no numérico se rechaza antes de llegar a la consulta (HTTP 404) y la consulta usa parámetro enlazado
'i'. Prueba: index.php?id=0 UNION SELECT 1-- - -> HTTP 404.
```

VUL-004 → SQL Injection en index.php – listado de cartas y categorías

Vulnerabilidad	SQL Injection en index.php – listado de cartas y categorías
Severidad	**Crítica**
OWASP Top 10	A03:2021 – Injection
OWASP ASVS v5.0.0	V1 Encoding and Sanitization
Descripción	Las consultas de cartas y de categorías concatenan $key y el id de carta. Tercer punto de inyección dentro de index.php.
URL	GET /index.php?id=<payload>
Archivo(s)	src/pnkSecurity/index.php
ISO/IEC 27001:2022	A.8.28
ISO/IEC 27034 (ASC)	ASC-10, ASC-14
Estado	Corregido – verificado

Evidencia (versión original — vulnerable)

```
Código vulnerable (revisión de código):
$sqlcartas="... WHERE cartas.restautantes_id = ".$key." and visible=1 ...";
```

Corrección

Commit Git: 294250b (rama mejoras)

Código original

```
$sqlcartas="SELECT cartas.id, ... FROM cartas
  WHERE cartas.restautantes_id = ".$key." and visible=1 AND eliminada IS NULL
  order by orden asc";
$sql_categorias="select id,nombre from categorias where visible=1
  and cartas_id='".$arraycartas[$i]["id"]."' AND eliminado IS NULL ...";
```

Código nuevo

```
// VUL004: consulta preparada.
$arraycartas = consulta_todas(
    "SELECT cartas.id, cartas.nombre FROM cartas
     WHERE cartas.restautantes_id = ? AND visible = 1 AND eliminada IS NULL
     ORDER BY orden ASC",
    'i', $key
);
$lista_categorias = consulta_todas(
    "SELECT id, nombre FROM categorias WHERE visible = 1 AND cartas_id = ? ...",
    'i', $carta['id']
);
```

Evidencia sin vulnerabilidad (versión corregida)

```
index.php?id=0 UNION SELECT 1-- - -> HTTP 404. Todas las consultas del archivo usan marcadores ? (grep de
mysqli_query en la rama mejoras: 0 resultados).
```

VUL-005 → SQL Injection en index.php – comentarios

Vulnerabilidad	SQL Injection en index.php – comentarios
Severidad	**Crítica**
OWASP Top 10	A03:2021 – Injection
OWASP ASVS v5.0.0	V1 Encoding and Sanitization
Descripción	La consulta que recupera los comentarios del restaurante concatena $key directamente.
URL	GET /index.php?id=<payload>
Archivo(s)	src/pnkSecurity/index.php
ISO/IEC 27001:2022	A.8.28
ISO/IEC 27034 (ASC)	ASC-10, ASC-14
Estado	Corregido – verificado

Evidencia (versión original — vulnerable)

```
$sqlcomentarios="select * from comentarios where id_restaurante=".$key;
```

Corrección

Commit Git: 294250b · dd0c4f8 (rama mejoras)

Código original

```
$sqlcomentarios="select * from comentarios where id_restaurante=".$key;
$resultcomentarios=mysqli_query(conectar(),$sqlcomentarios);
```

Código nuevo

```
// VUL005: consulta preparada. VUL026: solo los 50 más recientes.
$lista_comentarios = array_reverse(consulta_todas(
    "SELECT usuario, comentario FROM comentarios
     WHERE id_restaurante = ? ORDER BY Id DESC LIMIT 50",
    'i', $key
));
```

Evidencia sin vulnerabilidad (versión corregida)

```
index.php?id=0 UNION SELECT 1-- - -> HTTP 404 (rechazado).
```

VUL-006 → SQL Injection en mostrar_carrito.php

Vulnerabilidad	SQL Injection en mostrar_carrito.php
Severidad	**Crítica**
OWASP Top 10	A03:2021 – Injection
OWASP ASVS v5.0.0	V1 Encoding and Sanitization; V2 Validation
Descripción	La vista del carrito repite el patrón: $_GET['id'] concatenado sin validar en la consulta del restaurante (y además sin filtrar restaurantes eliminados).
URL	GET /mostrar_carrito.php?id=1 OR 1=1
Archivo(s)	src/pnkSecurity/mostrar_carrito.php
ISO/IEC 27001:2022	A.8.28
ISO/IEC 27034 (ASC)	ASC-10, ASC-14
Estado	Corregido – verificado

Evidencia (versión original — vulnerable)

```
original  mostrar_carrito.php?id=1 OR 1=1 -> HTTP 200 (inyección aceptada)
```

Corrección

Commit Git: 294250b (rama mejoras)

Código original

```
$key=$_GET['id'];
$sql_restorant="SELECT ... FROM restautantes INNER JOIN direcciones ...
  WHERE restautantes.id = ".$key;
$result_restorant=mysqli_query(conectar(),$sql_restorant);
```

Código nuevo

```
$key = entero_positivo($_GET['keyid'] ?? null);
if ($key === false) {
    responder_error(404, 'Restaurante no encontrado.');
}
$datos_restorant = consulta_fila(
    "SELECT restautantes.nombre, restautantes.id, restautantes.foto
     FROM restautantes INNER JOIN direcciones ...
     WHERE restautantes.id = ? AND restautantes.eliminado IS NULL",
    'i', $key
);
```

Evidencia sin vulnerabilidad (versión corregida)

```
corregida mostrar_carrito.php?keyid=1 OR 1=1 -> HTTP 404   |   ?keyid=1 -> HTTP 200
```

VUL-007 → SQL Injection en carrito.php (AJAX)

Vulnerabilidad	SQL Injection en carrito.php (AJAX)
Severidad	**Crítica**
OWASP Top 10	A03:2021 – Injection
OWASP ASVS v5.0.0	V1 Encoding and Sanitization; V4 API and Web Service
Descripción	El parámetro POST iditems, enviado por controladorajax.js, se concatena en la consulta que agrega un producto al carrito.
URL	POST /carrito.php op=1&iditems=0 UNION SELECT ...
Archivo(s)	src/pnkSecurity/carrito.php
ISO/IEC 27001:2022	A.8.28
ISO/IEC 27034 (ASC)	ASC-10, ASC-14, ASC-29
Estado	Corregido – verificado

Evidencia (versión original — vulnerable)

```
$sql="select id, nombre, precio from items where id=".$_POST['iditems'];
En la versión original el payload UNION agrega al carrito una fila con datos arbitrarios.
```

Corrección

Commit Git: 31c4312 (rama mejoras)

Código original

```
function insertar()
{
    $sql="select id, nombre, precio from items where id=".$_POST['iditems'];
    $result=mysqli_query(conectar(),$sql);
    $datos=mysqli_fetch_array($result);
    ...
}
```

Código nuevo

```
$iditem = entero_positivo($_POST['iditems'] ?? null);
if ($iditem === false || $restaurante === false) {
    responder_json(400, ['ok' => false, 'error' => 'Producto no válido.']);
}
$datos = consulta_fila(
    "SELECT items.id, items.nombre, items.precio FROM items ...
     WHERE items.id = ? AND items.visible = 1 AND items.eliminado IS NULL ...
       AND cartas.restautantes_id = ?",
    'ii', $iditem, $restaurante
);
```

Evidencia sin vulnerabilidad (versión corregida)

```
corregida op=1 iditems=0 UNION SELECT... (con token CSRF) -> HTTP 400
```

VUL-008 → SQL Injection en grcomentarios.php (INSERT)

Vulnerabilidad	SQL Injection en grcomentarios.php (INSERT)
Severidad	**Crítica**
OWASP Top 10	A03:2021 – Injection
OWASP ASVS v5.0.0	V1 Encoding and Sanitization
Descripción	usuario y comentario se concatenan en un INSERT. Permite inyección SQL y suplantar el nombre del autor del comentario.
URL	POST /grcomentarios.php usuario=...&comentario=...
Archivo(s)	src/pnkSecurity/grcomentarios.php
ISO/IEC 27001:2022	A.8.28
ISO/IEC 27034 (ASC)	ASC-10, ASC-14
Estado	Corregido – verificado

Evidencia (versión original — vulnerable)

```
$sql="INSERT INTO comentarios SET usuario='".$_POST['usuario']."',
      comentario='".$_POST['comentario']."',id_restaurante=".$_SESSION['id'];
Un apóstrofe en el comentario rompe la sentencia (error SQL en la versión original).
```

Corrección

Commit Git: e964933 (rama mejoras)

Código original

```
$sql="INSERT INTO comentarios SET usuario='".$_POST['usuario']."',
      comentario='".$_POST['comentario']."',id_restaurante=".$_SESSION['id'];
mysqli_query(conectar(),$sql);
```

Código nuevo

```
consulta(
    "INSERT INTO comentarios (usuario, usuario_id, comentario, id_restaurante)
     VALUES (?, ?, ?, ?)",
    'sisi', $_SESSION['nombre'], $_SESSION['usuario_id'], $comentario, $restaurante
);  // el autor sale de la sesión, no del formulario
```

Evidencia sin vulnerabilidad (versión corregida)

```
El comentario con <script> y apóstrofes se guarda literalmente (sin error SQL) y se muestra escapado:
&lt;script&gt;alert(1)&lt;/script&gt;
```

VUL-009 → Contraseñas almacenadas en texto plano

Vulnerabilidad	Contraseñas almacenadas en texto plano
Severidad	**Crítica**
OWASP Top 10	A02:2021 – Cryptographic Failures
OWASP ASVS v5.0.0	V6 Authentication; V11 Cryptography
Descripción	La tabla usuarios guarda la contraseña tal cual y el login la compara en SQL. Cualquier fuga de la BD (por ejemplo vía VUL-001..008) expone todas las credenciales.
URL	Tabla usuarios (src/Script_BD/pnk_security.sql) + POST /setup/procesalogin.php
Archivo(s)	src/Script_BD/pnk_security.sql, migracion_seguridad.sql, procesalogin.php
ISO/IEC 27001:2022	A.5.17, A.8.5, A.8.24
ISO/IEC 27034 (ASC)	ASC-07, ASC-12
Estado	Corregido – verificado

Evidencia (versión original — vulnerable)

```
original (texto plano):
  admin@gmail.com   -> admin01
  alondra@gmail.com -> alondra01
```

Corrección

Commit Git: f6070c8 (rama mejoras)

Código original

```
INSERT INTO `usuarios` VALUES
 (1,'Administrador','admin@gmail.com','admin01','1'),
 (2,'Alondra','alondra@gmail.com','alondra01','1');
-- login:  ... where email='...' and password='".$_POST['frmpassword']."'
```

Código nuevo

```
-- migracion_seguridad.sql: hash bcrypt (password_hash, cost 12)
UPDATE `usuarios` SET `password` = '$2y$12$M46C...' WHERE `email` = 'admin@gmail.com';
ALTER TABLE `usuarios` ADD UNIQUE KEY `uq_usuarios_email` (`email`);
// procesalogin.php
$valido = password_verify($password, $hash) && ...;
```

Evidencia sin vulnerabilidad (versión corregida)

```
corregida (hash bcrypt):
  admin@gmail.com   -> $2y$12$ODh076...  (60 chars)
  alondra@gmail.com -> $2y$12$mk.wpA...  (60 chars)
```

VUL-010 → Cross-Site Scripting (XSS) almacenado en comentarios

Vulnerabilidad	Cross-Site Scripting (XSS) almacenado en comentarios
Severidad	**Alta**
OWASP Top 10	A03:2021 – Injection (XSS)
OWASP ASVS v5.0.0	V1 Encoding and Sanitization (codificación de salida); V3 Web Frontend Security
Descripción	usuario y comentario se imprimen en el HTML sin htmlspecialchars(). Un script publicado como comentario se ejecuta en el navegador de cada visitante (robo de cookie, defacement).
URL	POST /grcomentarios.php -> GET /index.php?id=1 (sección Comentarios)
Archivo(s)	src/pnkSecurity/index.php, setup/setup.php
ISO/IEC 27001:2022	A.8.28
ISO/IEC 27034 (ASC)	ASC-10
Estado	Corregido – verificado

Evidencia (versión original — vulnerable)

```
Payload: <script>alert(document.cookie)</script>
original -> HTTP 200, SCRIPT SIN ESCAPAR (XSS) en index.php?id=1
```

Corrección

Commit Git: 294250b · 94ef3a9 (rama mejoras)

Código original

```
<b><?php echo $datoscomentarios['usuario'];?></b>
<br>
<?php echo $datoscomentarios['comentario'];?>
```

Código nuevo

```
function e($texto) {   // setup.php
    return htmlspecialchars((string) $texto, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
}
<b><?php echo e($datoscomentarios['usuario']);?></b>
<br>
<?php echo nl2br(e($datoscomentarios['comentario']));?>
// + CSP: script-src 'self' (VUL-022) como segunda barrera
```

Evidencia sin vulnerabilidad (versión corregida)

```
corregida: con sesión, <script> se guarda pero en el HTML aparece:
  &lt;script&gt;alert(1)&lt;/script&gt;
  [PASA] VUL-10 XSS -> script no reflejado
```

VUL-011 → Ausencia de token CSRF en el login

Vulnerabilidad	Ausencia de token CSRF en el login
Severidad	**Alta**
OWASP Top 10	A01:2021 – Broken Access Control (CSRF)
OWASP ASVS v5.0.0	V3 Web Frontend Security (protección CSRF)
Descripción	El formulario de login no lleva token anti-CSRF ni valida el origen: un sitio externo puede forzar inicios de sesión (login CSRF) y combinarlo con VUL-001.
URL	POST /setup/procesalogin.php (formulario en index.php)
Archivo(s)	index.php, setup/procesalogin.php, setup/setup.php
ISO/IEC 27001:2022	A.8.28
ISO/IEC 27034 (ASC)	ASC-09, ASC-10
Estado	Corregido – verificado

Evidencia (versión original — vulnerable)

```
<form class="form-inline" action="setup/procesalogin.php" method="post">
  ...  // sin campo csrf_token
```

Corrección

Commit Git: f6070c8 · 294250b · 94ef3a9 (rama mejoras)

Código original

```
<form class="form-inline" role="search" action="setup/procesalogin.php" method="post">
  <input type="text" name="frmusuario" placeholder="Usuario">
  <input type="password" name="frmpassword" placeholder="Contraseña">
</form>
```

Código nuevo

```
<input type="hidden" name="csrf_token" value="<?php echo e($csrf);?>">
// procesalogin.php
exigir_post();
validar_csrf();   // hash_equals($_SESSION['csrf_token'], $enviado) o 403
```

Evidencia sin vulnerabilidad (versión corregida)

```
corregida: POST sin token -> HTTP 403   |   GET -> HTTP 405
```

VUL-012 → Ausencia de token CSRF en comentarios

Vulnerabilidad	Ausencia de token CSRF en comentarios
Severidad	**Alta**
OWASP Top 10	A01:2021 – Broken Access Control (CSRF)
OWASP ASVS v5.0.0	V3 Web Frontend Security (protección CSRF)
Descripción	Un sitio externo puede hacer que un usuario autenticado publique comentarios (incluido el payload XSS de VUL-010) sin su consentimiento.
URL	POST /grcomentarios.php
Archivo(s)	src/pnkSecurity/grcomentarios.php
ISO/IEC 27001:2022	A.8.28
ISO/IEC 27034 (ASC)	ASC-09, ASC-10
Estado	Corregido – verificado

Evidencia (versión original — vulnerable)

```
<form action="grcomentarios.php" method="post"> ... </form>  // sin csrf_token
```

Corrección

Commit Git: e964933 · 294250b (rama mejoras)

Código original

```
include("setup/setup.php");
session_start();
$sql="INSERT INTO comentarios SET ...";   // sin verificar token
```

Código nuevo

```
iniciar_sesion();
exigir_post();
if (!usuario_autenticado()) { responder_error(403, 'Debe iniciar sesión para comentar.'); }
validar_csrf();
```

Evidencia sin vulnerabilidad (versión corregida)

```
corregida (con sesión): POST sin token -> HTTP 403
```

VUL-013 → Ausencia de token CSRF en el carrito (AJAX)

Vulnerabilidad	Ausencia de token CSRF en el carrito (AJAX)
Severidad	**Media**
OWASP Top 10	A01:2021 – Broken Access Control (CSRF)
OWASP ASVS v5.0.0	V3 Web Frontend Security; V4 API and Web Service
Descripción	Las llamadas AJAX que agregan, quitan o vacían el carrito no llevan token ni validan el origen; además vaciar el carrito destruía la sesión completa.
URL	POST /carrito.php op=1|2|3
Archivo(s)	src/pnkSecurity/carrito.php, js/controladorajax.js
ISO/IEC 27001:2022	A.8.28
ISO/IEC 27034 (ASC)	ASC-09, ASC-29
Estado	Corregido – verificado

Evidencia (versión original — vulnerable)

```
$.ajax({ type: "POST", url: 'carrito.php', data: "op=1&iditems="+id, ... });
// sin token CSRF
```

Corrección

Commit Git: 31c4312 (rama mejoras)

Código original

```
$.ajax({ type: "POST", url: 'carrito.php', data: "op=1&iditems="+id, ... });
// carrito.php
switch($_POST['op']) { ... case "3": eliminartodo(); }   // session_destroy()
```

Código nuevo

```
// controladorajax.js
headers: { 'X-CSRF-Token': $('meta[name="csrf-token"]').attr('content') }
// carrito.php
exigir_post();
validar_csrf();
switch ($_POST['op'] ?? '') { ... default: responder_json(400, ...); }
```

Evidencia sin vulnerabilidad (versión corregida)

```
corregida: op=1 SIN token CSRF -> HTTP 403
```

VUL-014 → Autorización solo en el cliente (comentarios sin sesión)

Vulnerabilidad	Autorización solo en el cliente (comentarios sin sesión)
Severidad	**Alta**
OWASP Top 10	A01:2021 – Broken Access Control
OWASP ASVS v5.0.0	V8 Authorization
Descripción	El formulario de comentarios solo se oculta en la interfaz al usuario anónimo, pero el endpoint acepta el POST de cualquiera.
URL	POST /grcomentarios.php (sin cookie de sesión)
Archivo(s)	src/pnkSecurity/grcomentarios.php
ISO/IEC 27001:2022	A.5.15, A.8.3
ISO/IEC 27034 (ASC)	ASC-08
Estado	Corregido – verificado

Evidencia (versión original — vulnerable)

```
original -> HTTP 200, comentario insertado sin iniciar sesión
```

Corrección

Commit Git: e964933 (rama mejoras)

Código original

```
include("setup/setup.php");
session_start();
// no existe if(isset($_SESSION['nombre']))
$sql="INSERT INTO comentarios SET usuario='".$_POST['usuario']."', ...";
```

Código nuevo

```
if (!usuario_autenticado()) {
    responder_error(403, 'Debe iniciar sesión para comentar.');
}
```

Evidencia sin vulnerabilidad (versión corregida)

```
corregida: POST a grcomentarios SIN sesión -> HTTP 403 (requiere login)
  [PASA] VUL-14 auth -> HTTP 403
```

VUL-015 → IDOR en el parámetro id (restaurante)

Vulnerabilidad	IDOR en el parámetro id (restaurante)
Severidad	**Alta**
OWASP Top 10	A01:2021 – Broken Access Control (IDOR)
OWASP ASVS v5.0.0	V8 Authorization (autorización a nivel de objeto)
Descripción	El id de restaurante recibido por GET se guarda directo en $_SESSION['id'] y se usa en todas las consultas siguientes sin validar que exista ni que esté activo. Permite enumerar recursos y dejar la sesión apuntando a restaurantes eliminados.
URL	GET /index.php?id=999 (y mostrar_carrito.php)
Archivo(s)	src/pnkSecurity/index.php, mostrar_carrito.php
ISO/IEC 27001:2022	A.5.15, A.8.3
ISO/IEC 27034 (ASC)	ASC-08
Estado	Corregido – verificado

Evidencia (versión original — vulnerable)

```
original  ?id=999 -> HTTP 200 (página vacía, la sesión queda con id=999)
```

Corrección

Commit Git: 294250b (rama mejoras)

Código original

```
$key=$_GET['id'];
$_SESSION['id']=$_GET['id'];   // sin validar existencia ni estado
```

Código nuevo

```
$key = entero_positivo($_GET['id'] ?? null);   // solo ^[0-9]{1,9}$ y > 0
if ($key === false) {
    responder_error(404, 'Restaurante no encontrado.');
}
$datos_restorant = consulta_fila("... WHERE restautantes.id = ? AND eliminado IS NULL", 'i', $key);
if ($datos_restorant === null) {
    responder_error(404, 'Restaurante no encontrado.');
}
$_SESSION['id'] = $key;   // solo después de comprobar el recurso
```

Evidencia sin vulnerabilidad (versión corregida)

```
corregida ?id=999 -> HTTP 404
```

VUL-016 → Lógica de negocio: agregar ítems ocultos/eliminados al carrito

Vulnerabilidad	Lógica de negocio: agregar ítems ocultos/eliminados al carrito
Severidad	**Media**
OWASP Top 10	A04:2021 – Insecure Design
OWASP ASVS v5.0.0	V2 Validation and Business Logic
Descripción	La consulta del carrito no filtra items.visible ni items.eliminado ni el restaurante actual: iterando ids se agregan productos ocultos, eliminados o de otro local.
URL	POST /carrito.php op=1&iditems=3 (ítem eliminado)
Archivo(s)	src/pnkSecurity/carrito.php
ISO/IEC 27001:2022	A.8.26, A.8.28
ISO/IEC 27034 (ASC)	ASC-02, ASC-14
Estado	Corregido – verificado

Evidencia (versión original — vulnerable)

```
original: op=1&iditems=3 -> el ítem eliminado se agrega al carrito
```

Corrección

Commit Git: 31c4312 (rama mejoras)

Código original

```
$sql="select id, nombre, precio from items where id=".$_POST['iditems'];
// falta visible=1, eliminado IS NULL y restaurante actual
```

Código nuevo

```
WHERE items.id = ? AND items.visible = 1 AND items.eliminado IS NULL
  AND categorias.visible = 1 AND categorias.eliminado IS NULL
  AND cartas.visible = 1 AND cartas.eliminada IS NULL
  AND cartas.restautantes_id = ?
// + máximo 50 ítems por carrito
```

Evidencia sin vulnerabilidad (versión corregida)

```
corregida: op=1 iditems=3 (item eliminado) -> HTTP 404
           op=1 iditems=5 (item visible)   -> HTTP 200
```

VUL-017 → Fijación de sesión (session fixation)

Vulnerabilidad	Fijación de sesión (session fixation)
Severidad	**Media**
OWASP Top 10	A07:2021 – Identification and Authentication Failures
OWASP ASVS v5.0.0	V7 Session Management
Descripción	Tras autenticar no se regenera el identificador de sesión: un atacante que fije el PHPSESSID de la víctima hereda su sesión autenticada.
URL	POST /setup/procesalogin.php
Archivo(s)	src/pnkSecurity/setup/procesalogin.php
ISO/IEC 27001:2022	A.8.5
ISO/IEC 27034 (ASC)	ASC-09
Estado	Corregido – verificado

Evidencia (versión original — vulnerable)

```
session_start();
$_SESSION['nombre']=$datos['nombre'];
// el PHPSESSID es el mismo antes y después del login
```

Corrección

Commit Git: f6070c8 · 94ef3a9 (rama mejoras)

Código original

```
session_start();
$_SESSION['nombre']=$datos['nombre'];
```

Código nuevo

```
session_regenerate_id(true);
$_SESSION['usuario_id'] = (int) $usuario['Id'];
$_SESSION['nombre']     = $usuario['nombre'];
$_SESSION['csrf_token'] = bin2hex(random_bytes(32));
// + session.use_strict_mode=1 y expiración por inactividad (30 min)
```

Evidencia sin vulnerabilidad (versión corregida)

```
corregida:
  ID antes:   52b73045b8ce0cbe30d55a8d37de6e1c
  ID después: 60a2f11b1f0dafd1015f64cf5d0dd188  -> el ID cambia
```

VUL-018 → Sin límite de intentos de login (fuerza bruta)

Vulnerabilidad	Sin límite de intentos de login (fuerza bruta)
Severidad	**Media**
OWASP Top 10	A07:2021 – Identification and Authentication Failures
OWASP ASVS v5.0.0	V6 Authentication (anti-automatización)
Descripción	No hay bloqueo, retardo ni contador de intentos fallidos: se pueden probar diccionarios de contraseñas sin restricción.
URL	POST /setup/procesalogin.php (repetido)
Archivo(s)	src/pnkSecurity/setup/procesalogin.php
ISO/IEC 27001:2022	A.8.5
ISO/IEC 27034 (ASC)	ASC-07
Estado	Corregido – verificado

Evidencia (versión original — vulnerable)

```
original: intentos ilimitados; ninguna respuesta cambia tras N fallos.
```

Corrección

Commit Git: f6070c8 · d57a953 (rama mejoras)

Código original

```
// ningún contador de intentos, CAPTCHA ni retardo
$result=mysqli_query(conectar(),$sql);
```

Código nuevo

```
const MAX_INTENTOS_CUENTA = 5;   // por IP + correo
const MAX_INTENTOS_IP     = 10;  // por IP
const VENTANA_MINUTOS     = 15;
$intentos = consulta_fila("SELECT COUNT(*) AS por_ip, COALESCE(SUM(email = ?), 0) AS por_cuenta
   FROM login_intentos WHERE ip = ? AND creado > (NOW() - INTERVAL 15 MINUTE)", 'ss', $email, $ip);
if (por_ip >= 10 || por_cuenta >= 5) -> 'Demasiados intentos fallidos...'
```

Evidencia sin vulnerabilidad (versión corregida)

```
corregida: tras 5 fallos, 6º intento con clave CORRECTA ->
  "Demasiados intentos fallidos. Intente nuevamente en 15 minutos."
```

VUL-019 → Credenciales de BD embebidas en el código (root sin clave)

Vulnerabilidad	Credenciales de BD embebidas en el código (root sin clave)
Severidad	**Media**
OWASP Top 10	A05:2021 – Security Misconfiguration
OWASP ASVS v5.0.0	V13 Configuration (gestión de secretos)
Descripción	setup.php conecta como root sin contraseña, versionado en Git. Cualquier fallo de la app opera con privilegios totales sobre MySQL.
URL	src/pnkSecurity/setup/setup.php (y GET /setup/setup.php)
Archivo(s)	src/pnkSecurity/setup/setup.php, deploy/user-data.sh
ISO/IEC 27001:2022	A.5.17, A.8.9, A.8.24
ISO/IEC 27034 (ASC)	ASC-13
Estado	Corregido – verificado

Evidencia (versión original — vulnerable)

```
function conectar(){
    $con=mysqli_connect("localhost","root","","pnk_security");
}
```

Corrección

Commit Git: 94ef3a9 · 3b4bca9 (rama mejoras)

Código original

```
function conectar()
{
    $con=mysqli_connect("localhost","root","","pnk_security");
    return $con;
}
```

Código nuevo

```
$con = new mysqli(
    config('PNK_DB_HOST') ?? 'localhost',
    config('PNK_DB_USER'),          // getenv() / config.local.php (fuera de Git)
    config('PNK_DB_PASS'),
    config('PNK_DB_NAME') ?? 'pnk_security'
);
-- AWS: usuario pnk_app con SELECT + INSERT mínimos, clave aleatoria
--      en /etc/apache2/pnk-secretos.conf (chmod 600, SetEnv)
```

Evidencia sin vulnerabilidad (versión corregida)

```
AWS: GET /setup/setup.php -> HTTP 403; la app conecta con pnk_app (no root);
el archivo .sql no es accesible desde la web.
```

VUL-020 → Validación de entradas insuficiente (causa raíz transversal)

Vulnerabilidad	Validación de entradas insuficiente (causa raíz transversal)
Severidad	**Media**
OWASP Top 10	A03:2021 – Injection
OWASP ASVS v5.0.0	V2 Validation and Business Logic
Descripción	Ningún parámetro (id, iditems, pos, op) se valida en tipo o formato antes de usarse. Es la causa raíz que habilita VUL-001..008.
URL	Todos los endpoints: index.php, mostrar_carrito.php, carrito.php, grcomentarios.php
Archivo(s)	setup/setup.php + endpoints
ISO/IEC 27001:2022	A.8.28
ISO/IEC 27034 (ASC)	ASC-10
Estado	Corregido – verificado

Evidencia (versión original — vulnerable)

```
$key=$_GET['id'];  // sin conversión ni validación
```

Corrección

Commit Git: 94ef3a9 · 294250b · 31c4312 (rama mejoras)

Código original

```
$key=$_GET['id'];
switch($_POST['op']) { ... }        // sin default
unset($_SESSION["carrito"][$_POST['pos']]);
```

Código nuevo

```
function entero_positivo($valor) {
    if (!is_string($valor) || !preg_match('/^[0-9]{1,9}$/', $valor)) return false;
    $n = (int) $valor;
    return $n > 0 ? $n : false;
}
switch ($_POST['op'] ?? '') { ... default: responder_json(400, ...); }
```

Evidencia sin vulnerabilidad (versión corregida)

```
corregida: op=9 (operación inválida) -> HTTP 400; id no numérico -> HTTP 404
```

VUL-021 → Errores técnicos expuestos en pantalla

Vulnerabilidad	Errores técnicos expuestos en pantalla
Severidad	**Baja**
OWASP Top 10	A05:2021 – Security Misconfiguration
OWASP ASVS v5.0.0	V16 Security Logging and Error Handling
Descripción	Ningún mysqli_query() valida su resultado; con display_errors activo un payload malformado muestra rutas absolutas y código.
URL	GET /index.php?id=1'
Archivo(s)	src/pnkSecurity/setup/setup.php
ISO/IEC 27001:2022	A.8.15, A.8.28
ISO/IEC 27034 (ASC)	ASC-27, ASC-28
Estado	Corregido – verificado

Evidencia (versión original — vulnerable)

```
original  ?id=1' -> Fatal error (ruta y línea del archivo visibles)
```

Corrección

Commit Git: 94ef3a9 (rama mejoras)

Código original

```
$result=mysqli_query(conectar(),$sql);
$datos=mysqli_fetch_array($result);   // sin verificar $result === false
```

Código nuevo

```
ini_set('display_errors', '0');
ini_set('log_errors', '1');
set_exception_handler(function ($ex) {
    error_log('[pnkSecurity] ' . get_class($ex) . ': ' . $ex->getMessage() ...);
    responder_error(500, 'Ocurrió un error inesperado. Intente nuevamente más tarde.');
});
```

Evidencia sin vulnerabilidad (versión corregida)

```
corregida ?id=1' -> HTTP 404 (sin traza técnica)
```

VUL-022 → Faltan cabeceras de seguridad y atributos de cookie

Vulnerabilidad	Faltan cabeceras de seguridad y atributos de cookie
Severidad	**Baja**
OWASP Top 10	A05:2021 – Security Misconfiguration
OWASP ASVS v5.0.0	V3 Web Frontend Security; V7 Session Management; V12 Secure Communication
Descripción	Sin CSP, X-Frame-Options, nosniff ni HSTS; la cookie PHPSESSID no tiene HttpOnly, Secure ni SameSite.
URL	Todas las respuestas HTTP
Archivo(s)	src/pnkSecurity/setup/setup.php
ISO/IEC 27001:2022	A.8.9, A.8.27
ISO/IEC 27034 (ASC)	ASC-06, ASC-09
Estado	Corregido – verificado

Evidencia (versión original — vulnerable)

```
original: sin cabeceras de seguridad; cookie PHPSESSID sin HttpOnly/SameSite/Secure
```

Corrección

Commit Git: 94ef3a9 (rama mejoras)

Código original

```
session_start();   // configuración por defecto de PHP, sin header() de seguridad
```

Código nuevo

```
header("Content-Security-Policy: default-src 'self'; script-src 'self'; ...");
header('X-Content-Type-Options: nosniff');
header('X-Frame-Options: DENY');
header('Strict-Transport-Security: max-age=31536000; includeSubDomains');  // HTTPS
session_set_cookie_params(['secure' => es_https(), 'httponly' => true,
                           'samesite' => 'Strict', 'path' => '/']);
```

Evidencia sin vulnerabilidad (versión corregida)

```
Content-Security-Policy: default-src 'self'; script-src 'self'; ...
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Strict-Transport-Security: max-age=31536000; includeSubDomains
Set-Cookie: PNKSESSID=...; path=/; secure; HttpOnly; SameSite=Strict
```

VUL-023 → Credenciales de prueba conocidas en producción y secretos en log

Vulnerabilidad	Credenciales de prueba conocidas en producción y secretos en log
Severidad	**Media-Alta**
OWASP Top 10	A07:2021 – Identification and Authentication Failures
OWASP ASVS v5.0.0	V6 Authentication; V13 Configuration
Descripción	Segunda auditoría: el despliegue usaba las mismas claves de laboratorio (publicadas en el README) y el log de aprovisionamiento registraba secretos con set -x.
URL	Login en https://174-129-56-84.sslip.io con las claves del README; /var/log/pnk-deploy.log
Archivo(s)	deploy/user-data.sh (+ commit 146508c en main: retira claves del README)
ISO/IEC 27001:2022	A.5.17, A.8.31, A.8.33
ISO/IEC 27034 (ASC)	ASC-13, ASC-24
Estado	Corregido – verificado en AWS (17/10/2026)

Evidencia (versión original — vulnerable)

```
README (main) publicaba las claves de admin@gmail.com y alondra@gmail.com, y user-data.sh dejaba DB_PASS
visible en /var/log/pnk-deploy.log (legible por otros usuarios).
```

Corrección

Commit Git: 534932c (rama mejoras)

Código original

```
exec > /var/log/pnk-deploy.log 2>&1      # con set -x: secretos al log
DB_PASS=$(openssl rand -hex 24)
mysql pnk_security < migracion_seguridad.sql  # hashes de claves conocidas
```

Código nuevo

```
install -m 600 /dev/null /var/log/pnk-deploy.log
set +x   # no escribir secretos en el log
for EMAIL in admin@gmail.com alondra@gmail.com; do
    CLAVE=$(openssl rand -base64 18 | tr -d '/+=')
    HASH=$(CLAVE="$CLAVE" php -r 'echo password_hash(getenv("CLAVE"), PASSWORD_BCRYPT, ["cost"=>12]);')
    mysql pnk_security -e "UPDATE usuarios SET password='${HASH}' WHERE email='${EMAIL}'"
    echo "${EMAIL}  ${CLAVE}" >> /root/pnk-credenciales.txt   # chmod 600
done
```

Evidencia sin vulnerabilidad (versión corregida)

```
Evidencia de configuración (código de despliegue). Verificación en instancia: login con admin01 falla; ls -l
/var/log/pnk-deploy.log -> -rw------- root.
```

VUL-024 → Bloqueo de cuentas ajenas (DoS) en el límite de intentos

Vulnerabilidad	Bloqueo de cuentas ajenas (DoS) en el límite de intentos
Severidad	**Media**
OWASP Top 10	A07:2021 – Identification and Authentication Failures
OWASP ASVS v5.0.0	V6 Authentication (anti-automatización)
Descripción	La primera versión del bloqueo (VUL-018) contaba fallos por IP O por correo: un atacante podía bloquear a otra persona fallando con su correo, y un login exitoso reiniciaba el contador de la IP.
URL	POST /setup/procesalogin.php
Archivo(s)	src/pnkSecurity/setup/procesalogin.php
ISO/IEC 27001:2022	A.8.5
ISO/IEC 27034 (ASC)	ASC-07
Estado	Corregido – verificado

Evidencia (versión original — vulnerable)

```
WHERE (ip = ? OR email = ?) ...   -> 5 fallos con el correo de la víctima la bloquean
DELETE FROM login_intentos WHERE ip = ? OR email = ?  -> reinicia el contador
```

Corrección

Commit Git: d57a953 (rama mejoras)

Código original

```
$intentos = consulta_fila("SELECT COUNT(*) AS total FROM login_intentos
   WHERE (ip = ? OR email = ?) AND creado > ...", 'ss', $ip, $email);
consulta("DELETE FROM login_intentos WHERE ip = ? OR email = ?", 'ss', $ip, $email);
```

Código nuevo

```
$intentos = consulta_fila("SELECT COUNT(*) AS por_ip, COALESCE(SUM(email = ?), 0) AS por_cuenta
   FROM login_intentos WHERE ip = ? AND creado > ...", 'ss', $email, $ip);
// solo se limpian los fallos de esta cuenta desde esta IP
consulta("DELETE FROM login_intentos WHERE ip = ? AND email = ?", 'ss', $ip, $email);
```

Evidencia sin vulnerabilidad (versión corregida)

```
corregida: el bloqueo se aplica a la IP atacante (5 por IP+cuenta, 10 por IP); la cuenta de la víctima sigue
accesible desde otra IP.
```

VUL-025 → Componentes JavaScript con vulnerabilidades conocidas

Vulnerabilidad	Componentes JavaScript con vulnerabilidades conocidas
Severidad	**Baja**
OWASP Top 10	A06:2021 – Vulnerable and Outdated Components
OWASP ASVS v5.0.0	V15 Secure Coding and Architecture (dependencias)
Descripción	jQuery 3.2.1 (CVE-2019-11358, CVE-2020-11022/11023) y Bootstrap 4.1.3 (CVE-2019-8331).
URL	GET /vendors/jquery/jquery-3.2.1.min.js
Archivo(s)	src/pnkSecurity/vendors/, index.php, mostrar_carrito.php
ISO/IEC 27001:2022	A.8.8, A.5.21
ISO/IEC 27034 (ASC)	ASC-15
Estado	Corregido – verificado

Evidencia (versión original — vulnerable)

```
original: /*! jQuery v3.2.1 | (c) JS Foundation  + Bootstrap 4.1.3
```

Corrección

Commit Git: 273af11 (rama mejoras)

Código original

```
<script src="vendors/jquery/jquery-3.2.1.min.js"></script>
<!-- Bootstrap v4.1.3 -->
```

Código nuevo

```
<script src="vendors/jquery/jquery-3.7.1.min.js"></script>
<!-- Bootstrap v4.6.2 -->
```

Evidencia sin vulnerabilidad (versión corregida)

```
corregida: jQuery v3.7.1 + Bootstrap v4.6.2;
GET /vendors/jquery/jquery-3.2.1.min.js -> HTTP 404 (ya no existe)
```

VUL-026 → Comentarios sin límite (spam / inundación)

Vulnerabilidad	Comentarios sin límite (spam / inundación)
Severidad	**Baja**
OWASP Top 10	A04:2021 – Insecure Design
OWASP ASVS v5.0.0	V2 Validation and Business Logic (anti-automatización)
Descripción	Un usuario autenticado podía publicar comentarios ilimitados y la página cargaba todos.
URL	POST /grcomentarios.php (repetido)
Archivo(s)	src/pnkSecurity/grcomentarios.php, index.php, migracion_seguridad_2.sql
ISO/IEC 27001:2022	A.8.26
ISO/IEC 27034 (ASC)	ASC-02
Estado	Corregido – verificado

Evidencia (versión original — vulnerable)

```
original: sin límite; SELECT * FROM comentarios sin LIMIT
```

Corrección

Commit Git: dd0c4f8 (rama mejoras)

Código original

```
consulta("INSERT INTO comentarios (usuario, comentario, id_restaurante) VALUES (?, ?, ?)", ...);
"SELECT usuario, comentario FROM comentarios WHERE id_restaurante = ? ORDER BY Id ASC"
```

Código nuevo

```
const MAX_COMENTARIOS = 5;
const VENTANA_COMENTARIOS_MIN = 10;
$recientes = consulta_fila("SELECT COUNT(*) AS total FROM comentarios
   WHERE usuario_id = ? AND creado > (NOW() - INTERVAL 10 MINUTE)", 'i', $_SESSION['usuario_id']);
if ((int) $recientes['total'] >= MAX_COMENTARIOS) { ... 'Ha publicado demasiados comentarios.' }
... ORDER BY Id DESC LIMIT 50
```

Evidencia sin vulnerabilidad (versión corregida)

```
corregida: se publican 6 comentarios -> 6º: "Ha publicado demasiados comentarios..."
Comentarios realmente guardados: 5 (máximo 5 por 10 min)
```

VUL-027 → Versión vulnerable expuesta indefinidamente en :8080

Vulnerabilidad	Versión vulnerable expuesta indefinidamente en :8080
Severidad	**Baja**
OWASP Top 10	A05:2021 – Security Misconfiguration
OWASP ASVS v5.0.0	V13 Configuration
Descripción	Para la evidencia comparativa se publica la rama main en :8080 (solo IP del tester). Si nadie la apagaba, quedaba expuesta de forma permanente.
URL	http://174.129.56.84:8080/index.php?id=1
Archivo(s)	deploy/user-data.sh, deploy/desplegar_y_evidenciar.ps1
ISO/IEC 27001:2022	A.8.9, A.8.31
ISO/IEC 27034 (ASC)	ASC-23, ASC-26
Estado	Corregido – verificado en AWS (17/10/2026)

Evidencia (versión original — vulnerable)

```
deploy/user-data.sh (3b4bca9) creaba el VirtualHost :8080 sin fecha de retiro.
```

Corrección

Commit Git: 534932c · c617a06 (rama mejoras)

Código original

```
a2ensite pnk-original     # queda publicado hasta que alguien lo apague
```

Código nuevo

```
# user-data.sh: la versión vulnerable se elimina sola a las 2 horas
pnk-retirar-original.timer  ->  OnBootSec=2h
    a2dissite pnk-original; rm -rf /var/www/original; DROP DATABASE pnk_original
# desplegar_y_evidenciar.ps1: apenas termina el testing se cierra 8080
ec2.revoke_security_group_ingress(GroupId=sg, IpPermissions=[regla 8080])
```

Evidencia sin vulnerabilidad (versión corregida)

```
desplegar_y_evidenciar.ps1 registra en resultado_testing.txt:
  Version original :8080 tras el testing -> cerrado (sin respuesta)
```

VUL-028 → Detección de HTTPS confiando en X-Forwarded-Proto del cliente

Vulnerabilidad	Detección de HTTPS confiando en X-Forwarded-Proto del cliente
Severidad	**Baja**
OWASP Top 10	A05:2021 – Security Misconfiguration
OWASP ASVS v5.0.0	V12 Secure Communication; V7 Session Management
Descripción	Tercera revisión: es_https() aceptaba la cabecera X-Forwarded-Proto enviada por cualquier cliente. Sin balanceador, un atacante podía hacer que el servidor creyera estar en HTTPS (cookie marcada Secure y HSTS emitidos sobre HTTP), ocultando una conexión no cifrada.
URL	Cualquier página: cabecera X-Forwarded-Proto: https sobre http://
Archivo(s)	src/pnkSecurity/setup/setup.php
ISO/IEC 27001:2022	A.8.9, A.8.27
ISO/IEC 27034 (ASC)	ASC-06, ASC-09
Estado	Corregido – verificar con test_seguridad.py

Evidencia (versión original — vulnerable)

```
curl -H 'X-Forwarded-Proto: https' http://<host>/index.php?id=1
-> Strict-Transport-Security y cookie 'secure' en una respuesta HTTP plana
```

Corrección

Commit Git: e534271 (rama mejoras)

Código original

```
function es_https()
{
    return (!empty($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !== 'off')
        || (($_SERVER['HTTP_X_FORWARDED_PROTO'] ?? '') === 'https');
}
```

Código nuevo

```
function es_https()
{
    if (!empty($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !== 'off') {
        return true;
    }
    // solo detrás de un balanceador configurado explícitamente
    return config('PNK_TRUST_PROXY') === '1'
        && ($_SERVER['HTTP_X_FORWARDED_PROTO'] ?? '') === 'https';
}
```

Evidencia sin vulnerabilidad (versión corregida)

```
La cabecera enviada por el cliente se ignora (PNK_TRUST_PROXY no está definido en AWS: Apache termina TLS
directamente).
```

VUL-029 → CSP con 'unsafe-inline' y fuentes externas

Vulnerabilidad	CSP con 'unsafe-inline' y fuentes externas
Severidad	**Baja**
OWASP Top 10	A05:2021 – Security Misconfiguration
OWASP ASVS v5.0.0	V3 Web Frontend Security
Descripción	La CSP permitía style-src 'unsafe-inline', lo que habilita inyección de CSS (exfiltración de datos con selectores) si aparece cualquier otra falla de salida. style.css importaba Google Fonts (dependencia externa no declarada). Faltaban COOP/CORP.
URL	Todas las respuestas HTML
Archivo(s)	setup/setup.php, index.php, mostrar_carrito.php, css/style.css
ISO/IEC 27001:2022	A.8.9, A.8.28
ISO/IEC 27034 (ASC)	ASC-06, ASC-10
Estado	Corregido – verificado en AWS (17/10/2026)

Evidencia (versión original — vulnerable)

```
Content-Security-Policy: ... style-src 'self' 'unsafe-inline'; ...
<div style="text-align: center;">  ·  @import url("https://fonts.googleapis.com/...")
```

Corrección

Commit Git: e534271 (rama mejoras)

Código original

```
header("Content-Security-Policy: default-src 'self'; script-src 'self';
        style-src 'self' 'unsafe-inline'; ...");
<div style="text-align: center;">
<a class="button" ... style="margin:0%">Seleccionar</a>
```

Código nuevo

```
header("Content-Security-Policy: default-src 'self'; script-src 'self';
        style-src 'self'; ...");
header('Cross-Origin-Opener-Policy: same-origin');
header('Cross-Origin-Resource-Policy: same-origin');
<div class="text-center">
<a class="button m-0" ...>Seleccionar</a>
```

Evidencia sin vulnerabilidad (versión corregida)

```
Prueba 'VUL-22/29 Cabeceras' de tests/test_seguridad.py: falla si la CSP contiene 'unsafe-inline' o si falta
nosniff, X-Frame-Options, HSTS o la cookie segura.
```

VUL-030 → Librerías obsoletas y archivos sin uso publicados en el webroot

Vulnerabilidad	Librerías obsoletas y archivos sin uso publicados en el webroot
Severidad	**Media**
OWASP Top 10	A06:2021 – Vulnerable and Outdated Components
OWASP ASVS v5.0.0	V15 Secure Coding and Architecture (dependencias)
Descripción	Aunque VUL-025 actualizó jQuery y Bootstrap, seguían publicados y descargables componentes viejos sin uso: Bootstrap 4.1.3 dentro de vendors/fontawesome (CVE-2019-8331), jQuery Validation 1.11.1 (ReDoS, CVE-2021-21252 / CVE-2022-31147), jQuery Form 3.32 (2013), Magnific Popup 1.1.0, ajaxChimp, skrollr, nouislider, scripts de contacto/mail, demos HTML, .DS_Store y style.map. Un atacante puede cargarlos desde la propia origen (bypass de CSP 'self' con gadgets de librerías viejas).
URL	GET /vendors/fontawesome/bootstrap.bundle.min.js · /js/jquery.validate.min.js · /css/style.map
Archivo(s)	src/pnkSecurity/js/, vendors/, css/ (423 archivos eliminados)
ISO/IEC 27001:2022	A.8.8, A.5.21
ISO/IEC 27034 (ASC)	ASC-15
Estado	Corregido – verificado en AWS (17/10/2026)

Evidencia (versión original — vulnerable)

```
vendors/fontawesome/bootstrap.bundle.min.js -> /*! Bootstrap v4.1.3 */  (HTTP 200)
js/jquery.validate.min.js -> /*! jQuery Validation Plugin - v1.11.1 - 3/22/2013 */  (HTTP 200)
```

Corrección

Commit Git: e71f454 (rama mejoras)

Código original

```
<script src="vendors/nice-select/jquery.nice-select.min.js"></script>
<script src="vendors/Magnific-Popup/jquery.magnific-popup.min.js"></script>
<script src="js/jquery.ajaxchimp.min.js"></script>
// main.js
$('#mc_embed_signup').find('form').ajaxChimp();
$(".hero-banner__video").magnificPopup({...});
```

Código nuevo

```
<script src="vendors/jquery/jquery-3.7.1.min.js"></script>
<script src="vendors/bootstrap/bootstrap.bundle.min.js"></script>   <!-- 4.6.2 -->
<script src="vendors/owl-carousel/owl.carousel.min.js"></script>
<script src="js/main.js"></script>
<script src="js/controladorajax.js"></script>
// webroot final: 4 librerías (jQuery 3.7.1, Bootstrap 4.6.2, Owl 2.3.4, Themify)
```

Evidencia sin vulnerabilidad (versión corregida)

```
Prueba 'VUL-19/25/30 Archivos': las rutas de las librerías retiradas responden 404.
```

VUL-031 → Fuerza bruta distribuida contra una cuenta y retención indefinida de IPs

Vulnerabilidad	Fuerza bruta distribuida contra una cuenta y retención indefinida de IPs
Severidad	**Baja**
OWASP Top 10	A07:2021 – Identification and Authentication Failures
OWASP ASVS v5.0.0	V6 Authentication (anti-automatización)
Descripción	Tras VUL-024 el límite era solo por IP: una botnet con muchas IP podía probar contraseñas contra una misma cuenta sin tope. Además login_intentos guardaba IPs (dato personal) para siempre.
URL	POST /setup/procesalogin.php desde muchas IP
Archivo(s)	src/pnkSecurity/setup/procesalogin.php
ISO/IEC 27001:2022	A.8.5, A.5.34
ISO/IEC 27034 (ASC)	ASC-07, ASC-11
Estado	Corregido – verificado por revisión de código

Evidencia (versión original — vulnerable)

```
WHERE ip = ? AND creado > ...   -> cada IP nueva parte con contador 0
login_intentos sin borrado de registros antiguos
```

Corrección

Commit Git: e3b4e54 (rama mejoras)

Código original

```
$intentos = consulta_fila("SELECT COUNT(*) AS por_ip, ... FROM login_intentos
   WHERE ip = ? AND creado > (NOW() - INTERVAL 15 MINUTE)", 'ss', $email, $ip);
```

Código nuevo

```
consulta("DELETE FROM login_intentos WHERE creado < (NOW() - INTERVAL 24 HOUR)");
$global = consulta_fila("SELECT COUNT(*) AS total FROM login_intentos
   WHERE email = ? AND creado > (NOW() - INTERVAL 15 MINUTE)", 's', $email);
if (... || (int) $global['total'] >= MAX_INTENTOS_GLOBAL) {   // 30
    registrar_evento('login_bloqueado', 'cuenta=' . $email);
```

Evidencia sin vulnerabilidad (versión corregida)

```
Máximo 30 intentos por cuenta cada 15 min desde cualquier IP (≈120/h). Con las claves aleatorias de 24
caracteres generadas al desplegar (VUL-023) un ataque en línea es impracticable. Las IP se eliminan a las 24
h.
```

VUL-032 → Registro insuficiente de eventos de seguridad

Vulnerabilidad	Registro insuficiente de eventos de seguridad
Severidad	**Baja**
OWASP Top 10	A09:2021 – Security Logging and Monitoring Failures
OWASP ASVS v5.0.0	V16 Security Logging and Error Handling
Descripción	Los ataques (CSRF rechazados, accesos denegados, logins fallidos, bloqueos) no quedaban registrados en ningún log, por lo que no era posible detectarlos ni investigarlos.
URL	procesalogin.php, grcomentarios.php, validar_csrf()
Archivo(s)	setup/setup.php, setup/procesalogin.php, grcomentarios.php
ISO/IEC 27001:2022	A.8.15, A.8.16
ISO/IEC 27034 (ASC)	ASC-27, ASC-31
Estado	Corregido – verificado en AWS (17/10/2026)

Evidencia (versión original — vulnerable)

```
responder_error(403, ...) sin ningún registro del evento
```

Corrección

Commit Git: e534271 · e3b4e54 (rama mejoras)

Código original

```
if (!hash_equals($_SESSION['csrf_token'], $enviado)) {
    responder_error(403, 'Solicitud rechazada: token CSRF inválido o ausente.');
}
```

Código nuevo

```
function registrar_evento($tipo, $detalle = '') {
    $limpio = preg_replace('/[\x00-\x1F\x7F]/', ' ', (string) $detalle);  // anti log injection
    error_log(sprintf('[pnkSecurity][seguridad] %s ip=%s ruta=%s %s', $tipo, $ip, $ruta,
                      mb_substr($limpio, 0, 200)));
}
registrar_evento('csrf_rechazado');      registrar_evento('login_fallido', ...);
registrar_evento('login_bloqueado', ...); registrar_evento('acceso_denegado', ...);
```

Evidencia sin vulnerabilidad (versión corregida)

```
En la instancia: grep '\[seguridad\]' /var/log/apache2/pnk_error.log muestra los eventos generados por
test_seguridad.py (sin contraseñas ni tokens).
```

VUL-033 → Aplicación publicada por HTTP plano si falla el certificado TLS

Vulnerabilidad	Aplicación publicada por HTTP plano si falla el certificado TLS
Severidad	**Media**
OWASP Top 10	A02:2021 – Cryptographic Failures
OWASP ASVS v5.0.0	V12 Secure Communication
Descripción	Si Let's Encrypt fallaba, user-data.sh solo mostraba un aviso y la aplicación quedaba en http:// con el formulario de login: las contraseñas viajarían sin cifrar.
URL	http://<ip-aws>.sslip.io/index.php?id=1 (cuando certbot falla)
Archivo(s)	deploy/user-data.sh
ISO/IEC 27001:2022	A.8.20, A.8.24
ISO/IEC 27034 (ASC)	ASC-06, ASC-12
Estado	Corregido – verificado en AWS (17/10/2026)

Evidencia (versión original — vulnerable)

```
certbot ... || echo "AVISO: no se pudo emitir el certificado; el sitio queda solo en HTTP"
```

Corrección

Commit Git: c617a06 (rama mejoras)

Código original

```
certbot --apache -d "$DOMINIO" --non-interactive --agree-tos \
        --register-unsafely-without-email --redirect \
    || echo "AVISO: no se pudo emitir el certificado; el sitio queda solo en HTTP"
```

Código nuevo

```
for intento in 1 2 3; do
    if certbot --apache -d "$DOMINIO" ... --redirect; then CERT_OK=1; break; fi
    sleep 60
done
if [ "$CERT_OK" != "1" ]; then     # fail-closed: nunca login por HTTP
    <VirtualHost *:80> RedirectMatch 503 ^/ </VirtualHost>
fi
# + FilesMatch deniega también .map .json .txt .sh .py config.*.php
```

Evidencia sin vulnerabilidad (versión corregida)

```
Con certificado: http:// redirige 301 a https://. Sin certificado: HTTP 503, la aplicación nunca se sirve sin
cifrar.
```

VUL-034 → Cabeceras de seguridad duplicadas (Apache + PHP)

Vulnerabilidad	Cabeceras de seguridad duplicadas (Apache + PHP)
Severidad	**Baja**
OWASP Top 10	A05:2021 – Security Misconfiguration
OWASP ASVS v5.0.0	V3 Web Frontend Security
Descripción	Detectado por tests/test_seguridad.py al desplegar en AWS: Apache agregaba X-Content-Type-Options, X-Frame-Options y Referrer-Policy además de las que ya envía PHP. El navegador recibe "nosniff, nosniff" y "DENY, DENY"; algunos navegadores y proxies descartan valores repetidos, con lo que la protección contra MIME sniffing y clickjacking puede quedar sin efecto (la CSP frame-ancestors 'none' sigue protegiendo el framing).
URL	https://174-129-56-84.sslip.io/index.php?id=1 (cabeceras de respuesta)
Archivo(s)	deploy/user-data.sh, src/pnkSecurity/.htaccess
ISO/IEC 27001:2022	A.8.9
ISO/IEC 27034 (ASC)	ASC-06, ASC-20
Estado	Corregido – verificado en AWS (17/10/2026)

Evidencia (versión original — vulnerable)

```
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
...
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
  [FALLA] VUL-22/29 Cabeceras -> FALTAN: nosniff, X-Frame-Options
```

Corrección

Commit Git: 62c29e9 (rama mejoras)

Código original

```
Header always set X-Content-Type-Options "nosniff"
Header always set X-Frame-Options "DENY"
Header always set Referrer-Policy "strict-origin-when-cross-origin"
```

Código nuevo

```
# PHP también envía estas cabeceras; se quita su copia y Apache deja una sola
Header unset X-Content-Type-Options
Header always set X-Content-Type-Options "nosniff"
Header unset X-Frame-Options
Header always set X-Frame-Options "DENY"
Header unset Referrer-Policy
Header always set Referrer-Policy "strict-origin-when-cross-origin"
```

Evidencia sin vulnerabilidad (versión corregida)

```
Aplicado en la instancia y recargado Apache: cada cabecera aparece una sola vez (también en las respuestas
403).
  [PASA] VUL-22/29 Cabeceras -> todas presentes
RESULTADO: 9 de 9 pruebas en PASA.
```

4. Evidencia de testing (sin vulnerabilidad)

El script tests/test_seguridad.py repite los ataques contra ambas versiones; tests/lab_local.sh levanta las dos ramas en bases de datos aisladas para reproducirlo sin AWS. Resultados completos en docs/evidencias/resultado_testing.txt (AWS) y docs/evidencias/resultado_local.txt (local, 05/10/2026). Despliegue en AWS del 17/10/2026: la primera ejecución detectó VUL-034 (cabeceras duplicadas); tras corregirla, 9 de 9 pruebas en PASA.

```
pip install requests
python tests/test_seguridad.py \
    --vulnerable http://174.129.56.84:8080 \
    --corregida  https://174-129-56-84.sslip.io
```

Análisis estático (SAST) de la versión corregida

```
semgrep scan --config p/php --config p/owasp-top-ten src/pnkSecurity
Ran 94 rules on 19 files: 0 findings.        (17/10/2026, rama mejoras)

GitHub Actions (.github/workflows/seguridad.yml) repite en cada push:
  php -l (sintaxis) · Semgrep (SAST) · Gitleaks (secretos en el repositorio)
```

Prueba	Original (main)	Corregida (mejoras)
VUL-001 Login SQLi ' OR '1'='1' -- -	Sesión abierta sin credenciales	Bloqueado
VUL-002..005 SQLi en id	HTTP 200	HTTP 404
VUL-006 SQLi mostrar_carrito	HTTP 200	HTTP 404
VUL-007 SQLi iditems	Inyección aceptada	HTTP 400
VUL-009 Contraseñas	Texto plano	bcrypt $2y$12$ (60 chars)
VUL-010 XSS almacenado	<script> sin escapar	<script> escapado
VUL-011/012/013 CSRF	Aceptado	HTTP 403 sin token
VUL-014 Comentario sin sesión	HTTP 200	HTTP 403
VUL-015 IDOR id=999	HTTP 200	HTTP 404
VUL-016 Ítem eliminado al carrito	Agregado	HTTP 404
VUL-017 ID de sesión tras login	Igual	Cambia
VUL-018/024 Fuerza bruta	Ilimitado	Bloqueo a los 5 fallos
VUL-019 GET /setup/setup.php	Accesible	HTTP 403 (AWS)
VUL-020 op=9	Sin control	HTTP 400
VUL-021 ?id=1'	Fatal error visible	HTTP 404 sin traza
VUL-022 Cabeceras	Ausentes	CSP, HSTS, XFO, nosniff, cookie segura
VUL-025 jQuery	3.2.1	3.7.1 (3.2.1 → 404)
VUL-026 6 comentarios seguidos	6 guardados	5 guardados, 6º rechazado
VUL-027 :8080 tras el testing	Publicado	Cerrado en el Security Group
VUL-028 X-Forwarded-Proto falso	HSTS/Secure sobre HTTP	Cabecera ignorada
VUL-029 CSP	style-src 'unsafe-inline'	Sin 'unsafe-inline' + COOP/CORP
VUL-030 Librerías viejas publicadas	HTTP 200	HTTP 404
VUL-031 Fuerza bruta distribuida	Sin tope por cuenta	30 por cuenta / 15 min
VUL-032 Eventos de seguridad	Sin registro	[seguridad] en el log
VUL-033 Falla de certificado	Login por HTTP	HTTP 503 (fail-closed)

5. AWS — versión sin vulnerabilidades

La versión corregida (rama mejoras) se despliega automáticamente en AWS EC2 con deploy/desplegar_aws.py y deploy/user-data.sh (commit 3b4bca9 + 534932c).

Elemento	Configuración
Servicio / región	EC2 Ubuntu 24.04 · us-east-1
URL corregida	https://174-129-56-84.sslip.io/index.php?id=1
Rama desplegada	mejoras
Red	Security Group: 80/443 públicos; 22 y 8080 solo IP del tester
TLS	HTTPS obligatorio (Let's Encrypt), http → https 301, HSTS
Base de datos	Usuario pnk_app con privilegios mínimos y clave aleatoria (no root)
Secretos	SetEnv en /etc/apache2/pnk-secretos.conf (600), fuera del webroot
Cuentas de prueba	Claves aleatorias por despliegue en /root/pnk-credenciales.txt (600)
Instancia	IMDSv2 obligatorio, disco EBS cifrado, unattended-upgrades
PHP / Apache	display_errors=Off, expose_php=Off, setup.php y .sql denegados
Versión original :8080	Solo para evidencia, IP del tester, se retira sola a las 2 h

Despliegue:

```
pip install boto3
# credenciales de AWS Academy en ~/.aws/credentials
python deploy/desplegar_aws.py --vulnerable --key vockey
# esperar 5-8 min y luego ejecutar tests/test_seguridad.py
```

Nota: AWS Academy recicla las instancias al cerrar el laboratorio, por lo que la IP cambia en cada despliegue. Reemplazar <IP-AWS> / <ip-aws> en este informe por la IP entregada por desplegar_aws.py.

6. Protección de datos personales

La aplicación trata datos personales básicos: correo y nombre de usuarios internos, nombre del autor de cada comentario y dirección IP en la tabla login_intentos. No trata datos sensibles. A la fecha del informe (17/10/2026) rige la Ley N° 19.628; la Ley N° 21.719 entra en vigencia el 01/12/2026, por lo que se considera como requisito de adecuación.

- Seguridad del tratamiento: las correcciones VUL-001..009 y VUL-019 eliminan las vías de acceso no autorizado a la tabla usuarios y protegen las contraseñas con bcrypt.
- Minimización: el comentario ya no recibe el nombre desde el formulario; se toma de la sesión.
- Retención: los intentos de login (IP y correo) se eliminan a las 24 horas (VUL-031); solo se muestran los 50 comentarios más recientes.
- Datos de prueba: el despliegue usa solo los datos ficticios del script SQL y genera claves aleatorias para las cuentas de prueba (VUL-023).
- Registro: los eventos de seguridad se registran sin contraseñas ni tokens (VUL-032).

7. Security Gate y riesgo residual

Campo	Resultado
Aplicación / versión	pnkSecurity · rama mejoras
Resultado	APROBADO
Hallazgos abiertos	0 de 34 (críticos 0 · altos 0 · medios 0 · bajos 0)
Controles automáticos	tests/test_seguridad.py (9 pruebas) + GitHub Actions: php -l, Semgrep (SAST) y Gitleaks (secretos) en cada push
Condición de liberación	desplegar_y_evidenciar.ps1 debe terminar con todas las pruebas en PASA y el puerto 8080 cerrado
Riesgo residual	Bajo y aceptado: el tope de 30 intentos por cuenta permite bloquear temporalmente (15 min) una cuenta con un ataque sostenido; se prefirió frente a permitir fuerza bruta distribuida.

Los capítulos ASVS se citan a nivel de capítulo (v5.0.0). El identificador exacto de cada requisito debe validarse contra la fuente oficial de OWASP ASVS antes de usarlo en una auditoría formal. Los ASC-xx son una taxonomía corporativa, no numeración oficial de ISO/IEC 27034.